

Benchmark dei tempi — piccola, media, grande

Misure di prestazione dell'intera pipeline sulle tre mappe di Dublino, prodotte da `scripts/benchmark.py` (dati grezzi in `sumo_comparison/benchmark.json`).

Nota sull'hardware. Le misure di tempo *reale* (wall) sono state raccolte in un ambiente molto modesto — **~3,5 GB di RAM, 1 core lento**, con il runtime Java `jdk4py`. Su un PC normale i tempi sono sensibilmente più bassi. I valori **indipendenti dalla macchina** (tempo di pianificazione riportato da ENHSP, nodi espansi, stati valutati, lunghezza del piano) sono quelli da confrontare per giudicare la difficoltà intrinseca dei problemi; i tempi wall vanno letti come ordine di grandezza e trend.

1. Risoluzione con ENHSP (problemi del repository)

Problemi `pddl_files/problem_<zona>.pddl`, dominio con turn-time e ritardo semaforico per movimento. Media di 3 esecuzioni, configurazione `-s aibr`.

Zona	Nodi	Archi	Semafori (rete)	Lunghezza piano	Nodi e spansi	Stati valutati	Tempo ENHSP	Wall (media)	Wall (min)
piccola	14	18	27	105	98	191	38 ms	0,71 s	0,49 s
media	50	93	97	212	195	389	238 ms	1,01 s	0,96 s
grande	120	206	453	177	163	330	141 ms	2,01 s	1,89 s

Osservazioni:

- Il **tempo di pianificazione vero e proprio è minuscolo** (38–238 ms). La differenza col tempo wall (0,5–2 s) è quasi tutta **avvio della JVM** e parsing: ogni esecuzione lancia un processo Java da zero, come fa `run.py`.
- Il problema **più impegnativo per la ricerca non è il più grande**: media (50 nodi) espande più nodi di grande (120), perché il suo percorso ottimo è più lungo (212 azioni contro 177) e attraversa più semafori. Il numero di nodi del grafo conta meno della struttura del percorso.
- `grande` ha molti più semafori nella rete (453) ma il sottografo da 120 nodi ne usa solo una piccola parte.

2. Pipeline SUMO (estrazione + iniezione)

Tempi degli script `extract_sumo_data.py` (lettura del `net.xml`, calcolo ritardi e turn) e `inject_signal_plan.py` (scrittura dell'additional-file).

Zona	Semafori	Estrazione	Iniezione
------	----------	------------	-----------

piccola	27	0,36 s	0,09 s
media	97	2,76 s	0,22 s
grande	453	5,40 s	0,39 s

L'estrazione scala con la dimensione della rete (è dominata dal calcolo degli angoli di svolta sulla geometria di tutte le corsie), ma resta comunque nell'ordine dei secondi. L'iniezione è quasi istantanea.

3. Scalabilità della risoluzione (numero di nodi)

Zona media rigenerata a dimensioni crescenti, stesso dominio, stessa configurazione. Serve a capire fino a dove ENHSP regge.

Nodi	Archi	Wall	Nodi espansi
50	93	1,5 s	195
120	235	2,7 s	76
200	420	6,0 s	348
300	632	9,4 s	477
400	839	15,7 s	559

Il tempo cresce **più che linearmente** con i nodi: il costo è dominato dal **grounding**, non dalla ricerca. Sia `start-move` sia `signal-delay` hanno tre argomenti (`prev` , `from` , `to`), quindi il numero di istanze da generare cresce col cubo dei nodi. Su questo hardware modesto (1 core) oltre i ~400 nodi la risoluzione diventa impraticabile e verso i ~939 nodi (media completa) la JVM esaurisce la memoria con heap piccola. Su un PC normale con heap 6 GB il tetto è più alto, ma la crescita resta cubica: per solve rapidi conviene restare sotto i ~400 nodi.

4. Guadagno misurato in simulazione (punto 4)

Dal confronto SUMO baseline vs semafori ottimizzati (`compare_sumo.py`), attesa media ai semafori su ~45 veicoli con le stesse rotte nei due scenari:

Zona	Veicoli	Attesa baseline	Attesa ottimizzata	Δ
piccola	46	3,2 s	1,0 s	-68,5 %
media	45	44,5 s	34,2 s	-23,1 %
grande	43	99,1 s	109,1 s	+10,1 %

Su piccola e media l'ottimizzazione riduce l'attesa; su grande la peggiora, perché la formula di Webster usata per ottimizzare non modella le code fra incroci (dettagli nel documento di confronto, sezione 10).

In sintesi

- **Risolvere i problemi del repository è veloce:** ENHSP impiega da 38 ms (piccola) a 238 ms (media) di pianificazione pura; con l'avvio della JVM si resta sotto i 2 secondi anche su grande.
- **Il collo di bottiglia è la dimensione, non la difficoltà del percorso:** il tempo cresce col cubo dei nodi per via del grounding delle azioni a tre argomenti. Fino a ~300–400 nodi la risoluzione è comoda; oltre diventa pesante.
- **La pipeline SUMO è leggera:** estrazione dati e iniezione del piano si misurano in secondi anche su grande (453 semafori).
- **L'ottimizzazione semaforica paga in simulazione** su piccola (–68 % di attesa) e media (–23 %), ma non su grande (+10 %): la verifica del punto 4 è ciò che permette di distinguere i due casi.

I tempi wall sopra sono di un ambiente molto lento (1 core): su un portatile normale vanno divisi grossomodo per un fattore 3–5. Per rieseguire i benchmark:

```
python scripts/benchmark.py          # ENHSP + pipeline + scalabilità
python scripts/benchmark.py --no-scaling # solo problemi del repository
python scripts/benchmark.py --runs 5   # più ripetizioni per stabilità
```